



Ottimizzazione 3 - *Regole di base*

Principi della Programmazione Parallela
CdL Informatica - A.A. 2025/2026



Ottimizzazione

Outline



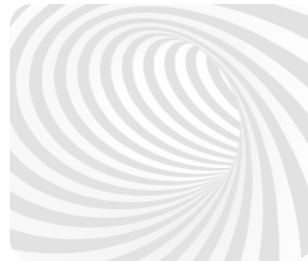
**Evitare le
inefficienze
evitabili**



Salti
Condizionali



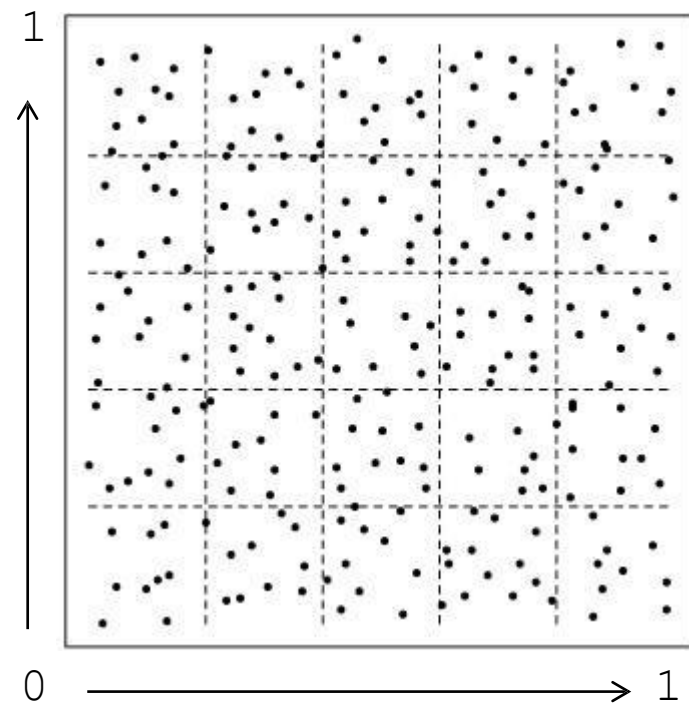
Pipeline



Cicli

Ai fini di impostare un esempio, supponiamo che:

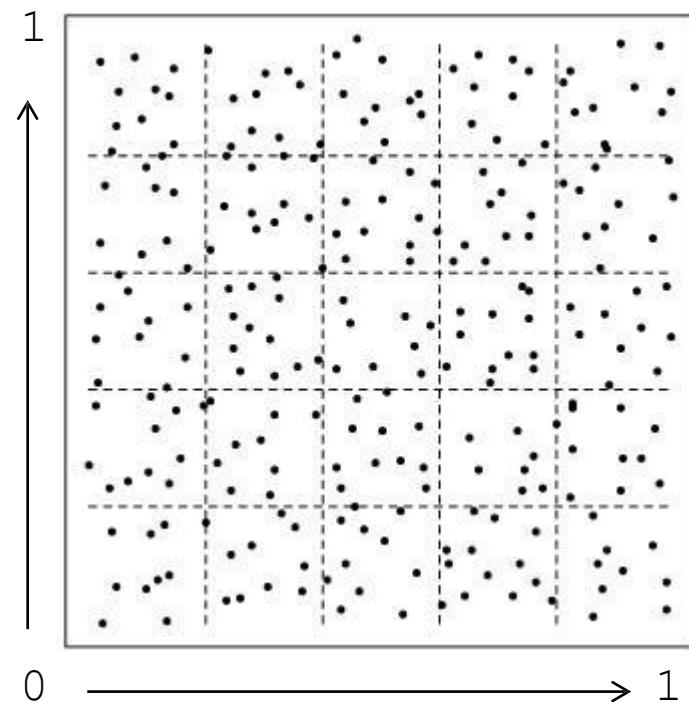
1) Supponiamo di avere una distribuzione di punti dati casuali su un piano 2D che suddividiamo in sotto-regioni utilizzando una griglia.



Ai fini di impostare un esempio, supponiamo che:

1) Supponiamo di avere una distribuzione di punti dati casuali su un piano 2D che suddividiamo in sotto-regioni utilizzando una griglia.

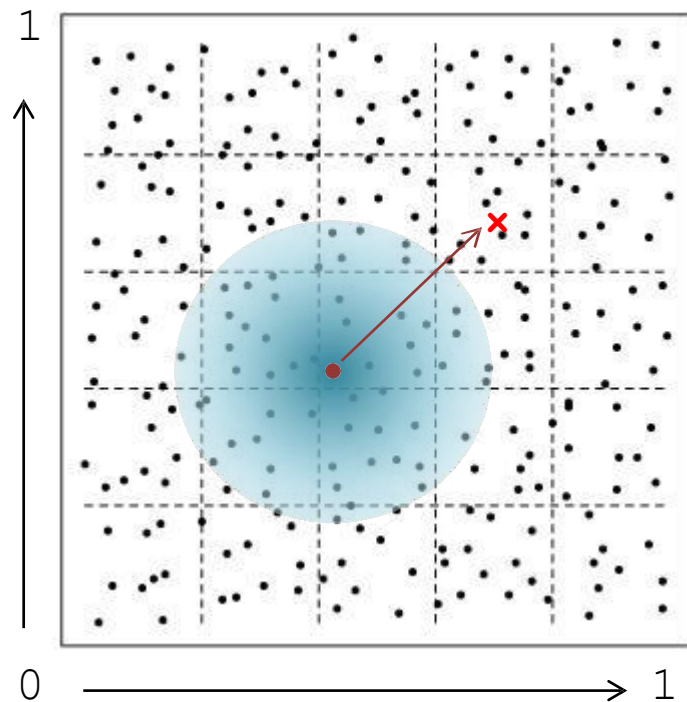
2) Per ogni punto p , vogliamo selezionare tutte le celle della griglia il cui centro è più vicino a p di un dato raggio r , ed eseguire alcune operazioni in base al risultato della nostra ricerca.



Ai fini di impostare un esempio, supponiamo che:

1) Supponiamo di avere una distribuzione di punti dati casuali su un piano 2D che suddividiamo in sotto-regioni utilizzando una griglia.

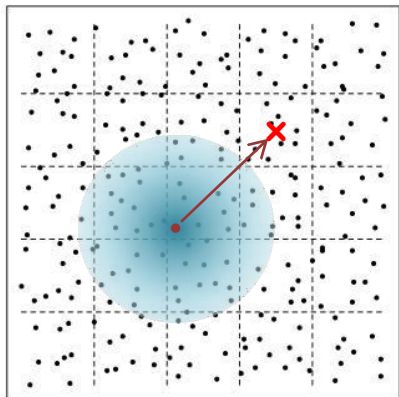
2) Per ogni punto p , vogliamo selezionare tutte le celle della griglia il cui centro è più vicino a p di un dato raggio r , ed eseguire alcune operazioni in base al risultato della nostra ricerca.



Potremmo considerare di usare un loop annidato come questo. C'è qualcosa che cambieresti?

```
for(p = 0; p < Np; p++)
  for(i = 0; i < Ng; i++)
    for(j = 0; j < Ng;
        j++)
      for(k = 0; k < Ng; k++)
        {
          dist = sqrt((x[p] - (double)i/Ng + half_size, 2) +
                     pow(y[p] - (double)j/Ng + half_size, 2) +
                     pow(z[p] - (double)k/Ng + half_size, 2));

          if(dist < R)
            fai qualcosa;
        }
```





(1) Evitare Chiamate a Funzioni Costose



Alcune chiamate a funzioni sono particolarmente costose, tra cui `sqrt()`. Cercare di evitarle se possibile (e.g., confrontando la distanza al quadrato `dist2 < R2`).

```
for(p = 0; p < Np; p++)  
    for(i = 0; i < Ng; i++)  
        for(j = 0; j < Ng;  
            j++)  
            for(k = 0; k < Ng; k++)  
                { dist2 = pow(x[p] - (double)i/Ng + half_size, 2) +  
                    pow(y[p] - (double)j/Ng + half_size, 2) +  
                    pow(z[p] - (double)k/Ng + half_size, 2));  
  
                if(dist2 < R2)  
                    fai qualcosa;  
                }
```

Essendo una chiamata esterna
ha un costo non indifferente





(1) Evitare Chiamate a Funzioni Costose



Alcune chiamate a funzioni sono particolarmente costose, tra cui `sqrt()` e `pow()`. Cercare di evitarle se possibile.

Same as before

```
for(p = 0; p < Np; p++)  
  
    for(i = 0; i < Ng; i++)  
        for(j = 0; j < Ng;  
            j++)  
            for(k = 0; k < Ng; k++)  
                { dx = x[p] - (double)i/Ng + half_size;  
                  dy = y[p] - (double)j/Ng + half_size;  
                  dz = z[p] - (double)k/Ng + half_size;  
  
                  dist2 = dx*dx + dy*dy + dz*dz;  
                  if(dist2 < R2)  
                      fai qualcosa;  
                }
```





(1) Evitare Chiamate a Funzioni Costose



Alcune chiamate a funzioni sono particolarmente costose, tra cui `sqrt()` e `pow()` e la divisione in virgola mobile. Cercare di evitarle se possibile.

```
for(p = 0; p < Np; p++)  
    for(i = 0; i < Ng; i++)  
        for(j = 0; j < Ng;  
            j++)  
            for(k = 0; k < Ng; k++)  
                {  
                    dx = x[p] - (double)i * Ng_inv + half_size;  
                    dy = y[p] - (double)j * Ng_inv + half_size;  
                    dz = z[p] - (double)k * Ng_inv +  
half_size;  
                    dist2 = dx*dx + dy*dy + dz*dz;  
                    if(dist2 < R2)  
                        }    fai qualcosa con sqrt(dist2);
```

la divisione ha un costo poiché
ogni volta che la chiami, devi trovare l'inverso



loop.3.c



(1) Evitare Chiamate a Funzioni Costose



```
(double)<i,j,k> * Ng_inv + half_size
```

viene eseguita N^3+N^2+N volte,
ritornando sempre lo stesso valore.

L' Hoisting salverebbe

$N(N^2+N^1+1)$ mul, add e mem accessi.

Si può fare meglio pre-calcolando i valori
rilevanti in un array:

```
double ijk[Ng];  
for(i = 0; i < Ng; i++)  
    ijk[i] = i * Ng_inv + half_size
```

```
for(p = 0; p < Np; p++)
```

```
    for(i = 0; i < Ng; i++)  
        for(j = 0; j < Ng; j++)  
            for(k = 0; k < Ng; k++)  
            {  
                dx = x[p] - (double)i * Ng_inv + half_size;  
                dy = y[p] - (double)j * Ng_inv + half_size;  
                dz = z[p] - (double)k * Ng_inv + half_size;  
  
                dist2 = dx*dx + dy*dy + dz*dz;  
                if(dist2 < R2)  
                    fai qualcosa con  
                    sqrt(dist2);  
            }
```



(2) Hoisting delle espressioni



esempio di
Hoisting in cui
i calcoli
indipendenti
dai loop
interni
vengono
spostati verso
l'alto.

```
for(i = 0; i < Ng; i++) {  
    dx2 = x[p] - (double)i * Ng_inv + half_size;  
    dx2 = dx2*dx2; ← niente pow
```

```
    for(j = 0; j < Ng; j++) {  
        dy2 = y[p] - (double)j * Ng_inv + half_size;  
        dy2 = dy2*dy2;  
        dist2_xy = dx2 + dy2;  
        for(k = 0; k < Ng; k++) {  
            dz = z[p] - (double)k * Ng_inv + half_size;  
            dist2 = dist2_xy + dz*dz;  
            if(dist2 < Rmax2)  
                fai qualcosa con sqrt(dist2);        }  
    }
```

*poiché i risultati non dipendono dal ciclo
interno, non ha senso tenerli lì dato che li ricalcoli k volte*



loop. 4.c

11



(2) *Hoisting* delle espressioni



Si può fare ancora meglio
pre-calcolando i valori:

```
double ijk[Ng];  
for(i = 0; i < Ng; i++)  
    ijk[i] = i * Ng_inv + half_size
```

```
for(i = 0; i < Ng; i++) {  
    dx2 = x[p] - ijk[i];  
    dx2 = dx2*dx2;  
  
    for(j = 0; j < Ng; j++) {  
        dy2 = y[p] - ijk[j];  
        dist2_xy = dx2 + dy2*dy2;  
  
        for(k = 0; k < Ng; k++) {  
            dz = z[p] - ijk[k];  
            dist2 = dist2_xy + dz*dz;  
            if(dist2 < Rmax2)  
                fai qualcosa con sqrt(dist2);  
        } } }  
}
```





(3) Chiarire lo Scope delle Variabili

*dichiara le variabili dove servono
così il compilatore sa quando sono utili*

Tutte queste sono
variabili locali e non
è necessario
utilizzarle al di fuori.

Questo potrebbe
aiutare il
compilatore nell'
ottimizzare lo stack e
l'uso dei registri.

```
for(int i = 0; i < Ng; i++) {  
    double dx2 = x[p] - (double)i * Ng_inv + half_size;  
    dx2 *= dx2;  
  
    for(j = 0; j < Ng; j++) {  
        double dy2 = y[p] - (double)j * Ng_inv + half_size;  
        double dist2_xy = dx2 + dy2*dy2;  
  
        for(k = 0; k < Ng; k++) {  
            double dz = z[p] - (double)k * Ng_inv + half_size;  
            double dist2 = dist2_xy + dz*dz;  
  
            if(dist2 < Rmax2)  
                fai qualcosa con sqrt(dist2);  
        }  
    }  
}
```



(4) Suggerisci cosa è importante



Queste variabili vengono spesso calcolate e riutilizzate in seguito.

Potrebbe essere utile mantenere un registro dedicato a loro.

Nota: questo è un suggerimento, il compilatore, dopo aver analizzato il codice, potrebbe decidere diversamente.

```
double register Ng_inv = 1.0 / Ng;
for(int i = 0; i < Ng; i++) {
    double dx2 = x[p] - (double)i * Ng_inv + half_size;
    dx2 *= dx2;

    for(j = 0; j < Ng; j++) {
        double dy2 = y[p] - (double)j * Ng_inv + half_size;
        dy2 *= dy2;
        double register dist2_xy = dx2 + dy2;

        for(k = 0; k < Ng; k++) {
            double register dz = z[p] - (double)k * Ng_inv -
            ...;
            double register dist2 = dist2_xy + dz*dz;
            if(dist2 < Rmax2)
                fai qualcosa con sqrt(dist2);
        } } }
```



loop.5.c

Nota: l'importanza di essere coscienti

Il tuo compilatore è in grado di gestire la maggior parte degli aspetti che abbiamo visto sopra (tranne uno, ovviamente: l'utilizzo di chiamate di funzioni costose, come `pow` e `sqrt`).

Lo vedremo meglio quando parleremo di cicli e rami.

Tuttavia, questo non è un buon motivo per non scrivere un codice decente fin dall'inizio.



Attenzione!



Non supporre che il tuo compilatore sia sempre in grado di riorganizzare i calcoli – come evitare chiamate costose o usare espressioni matematicamente-equivalenti più convenienti – ogni volta. Potrebbe essere in grado di farlo per i calcoli interi ma non lo farà per i calcoli in virgola mobile.

La ragione è semplice, ed è legata al fatto che su un sistema digitale la matematica non è sempre come sulla lavagna:

La matematica intera ($+$ e $\times$) in complemento a 2 è commutativa e associativa.

La matematica in virgola mobile ($+$ e $\times$) è commutativa tra 2 operandi ma non è **mai** associativa.



Attenzione!

Infatti, se a , b e c sono numeri FP,

$$(a + b) + c \neq a + (b + c)$$

a causa della natura stessa della rappresentazione in virgola mobile in un sistema digitale (con un numero finito di cifre).

Il problema è legato al numero limitato di cifre disponibili per rappresentare il numero che, a sua volta, limita la precisione.



Attenzione!

Supponiamo di avere 3 cifre di precisione per la mantissa. Per semplicità, consideriamo una base 10 (quindi ogni singola cifra è compresa in $[0..9]$).

Allora si verifica quanto segue:

$$1.00 + 0.01 = 1.01$$

$$1.00 + 0.001 = 1.00$$

L'ultimo è vero perché, sebbene possiamo rappresentare 0.001 (è 1.00 con un esponente -3) la somma di 1.00 e 0.001 è oltre la nostra precisione: 1.001 richiederebbe 4 cifre. Di conseguenza, non siamo in grado di distinguerlo da 1.00.



Attenzione!

Quindi di nuovo:

Questi
sommano
a 0.01

0.001 +

0.001 +

0.001 +

0.001 +

0.001 +

0.001 +

0.001 +

0.001 +

0.001 +

0.001 +

1.00

=

1.01

1.00 +

0.001 +

0.001 +

0.001 +

0.001 +

0.001 +

0.001 +

0.001 +

0.001 +

0.001 +

0.001 =

1.00

Ognuna
di queste
operazio
ni risulta
in 1.00

Quindi, il compilatore NON è libero di rimescolare l'ordine delle operazioni in virgola mobile,

...anche se una formulazione matematicamente-equivalente, diversa da quella che hai codificato, sarebbe più performante.



(5) Non ripetere controlli non necessari



Ti aspetti una
grande
performance da
questo codice?

Se no, perché?

```
char * find_char_in_string( char *string, char c )
{
    int i = 0;
    while ( i < strlen(string) )
        if( string[i] == c )
            break;
        else
            i++;

    if( i < strlen(string) )
        return &string[i];
    else
        return NULL;
}
```

é un controllo
inutile che si
ripete!
Poi stop subito dopo
il while



(5) Non ripetere controlli non necessari

```
char * find_char_in_string( char *string, char c )
{
    int i = 0;
    while ( i < strlen(string) )
    {
        if( string[i] == c )
        {
            break;
        }
        else
        {
            i++;
        }
    }

    if( i < strlen(string) )
    {
        return &string[i];
    }
    else
    {
        return NULL;
    }
}
```

chiamato a funz
di libreria
ripetuto più volte!

Ci sono diversi dettagli che abbassano la performance, cioè il Cycles Per Element, di questo loop.

Quello su cui voglio attirare la tua attenzione è la chiamata ripetuta alla funzione `strlen()`.

Ti aspetti che la stringa cambi mentre la stai scorrendo?

Immagino di no, ma il compilatore non lo sa e non ha modo di capirlo tramite l'analisi del codice. Inoltre, la memoria puntata da `string` potrebbe essere modificata altrove tra due iterazioni.



(5) Non ripetere controlli non necessari

Questo semplicissimo cambiamento ti farà risparmiare molto in termini di CPE

```
char * find_char_in_string( char *string, char c )
{
    int i = 0;
    int len = strlen(string);
    while ( i < len )
        if( string[i] == c )
            break;
        else
            i++;

    if( i < strlen(string) )
        return &string[i];
    else
        return NULL;
}
```




(5) Non ripetere controlli non necessari

Per una serie di ragioni, questa versione è ancora più efficiente della precedente.

Sai dire perché?

```
char * find_char_in_string( char *string, char c )
{
    char *pos = string;
    while( ( *pos != '\0' ) && ( *pos != c ) )
        pos++;

    return ( *pos == '\0' ? NULL : pos );
}
```

*usiamo solo aritmetica
dei puntatori.*



(5) Evitare riferimenti di memoria non necessari

Questo semplice loop per la riduzione di un array accumula i risultati parziali dereferenziando il puntatore *sum* ad ogni iterazione.

```
void reduce_vector( int n, double *array, double *sum )  
{  
    for ( int i = 0; i < n; i++ )  
        *sum += array[i];  
    return;  
}
```

↑
vai ad aggiornare
una somma esterna tante
volte facendo loopi accessi
esterni



(5) Evitare riferimenti di memoria non necessari

Questo semplice loop per la riduzione di un array accumula i risultati parziali dereferenziando il puntatore sum più volte.

(asm ottenuto con -O1)

```
void reduce_vector( int n, double *array, double *sum )
{
    for ( int i = 0; i < n; i++ )
        *sum += array[i];
    return;
}
```

```
.L3:
    movsd xmm0, QWORD PTR [rdx]
    addsd xmm0, QWORD PTR [rax]
    movsd QWORD PTR [rdx], xmm0
    add    rax, 8
    cmp    rax, rcx
    jne    .L3
```

```
movsd xmm0, value of *sum
addsd  xmm0, value of *array
movsd  address of sum, xmm0
add    rax, 8      ( array++ )
cmp    rax, n
```



(5) Evitare riferimenti di memoria non necessari

Introducendo un accumulatore locale separato si risparmianno accessi alla memoria

(asm ottenuto con -O1)

```
void reduce_vector( int n, double *array, double *sum )
{
    double temp = 0;
    for ( int i = 0; i < n; i++ )
        temp += array[i];
    *sum = temp;
    return;
}
```

aggiornab solo alla fine!

```
.L11:
addsd xmm0, QWORD PTR [rax]
add    rax, 8
cmp    rax, rdx
jne    .L11
```

addsd xmm0, value of *array

add rax, 8 (array++)

cmp rax, rdx (array with
end-of-array)

Riepilogando

Regola 1: evitate le funzioni matematiche costose quando non sono necessarie.

Regola 2: fate hoisting delle espressioni invarianti. Se un calcolo produce lo stesso risultato per tutte le iterazioni di un loop, spostatelo fuori dal loop. Farlo esplicitamente rende il codice piu' leggibile e aiuta il compilatore.

Regola 3: dichiarate le variabili nel loro scope minimo. Questo migliora la leggibilita', aiuta il compilatore ad allocare registri e stack in modo ottimale, e vi protegge da bug sottili.

Regola 4: usate register per suggerire al compilatore le variabili critiche del loop interno, con la consapevolezza che e' un suggerimento non vincolante.

Regola 5: non ripetete calcoli invarianti nella condizione del loop, e usate variabili locali come accumulatori invece di dereferenziare puntatori ad ogni iterazione.

Mi Esercito

Adesso provate voi ad ottimizzare il codice presente nell'esempio `loop.0.c` [slide #6] apportando le migliorie:

(1) Evitare Chiamate a Funzioni Costose

`loop.1.c` [slide #7]

`loop.2.c` [slide #8]

`loop.3.c` [slide #9]

(2) Hoisting delle espressioni

`loop.4.c` [slide #11]

`loop.6.c` [slide #12]

(4) Suggerisci cosa è importante

`loop.5.c` [slide #14]

Mi Esercito

Compilare con:

```
gcc -o loop.n loop.n.c -lm
```

ed eseguire confrontando i tempi.

Poi verificare cosa succede con la versione ottimizzata dal compilatore:

```
gcc -O3 -march=native -o loop.n.0 loop.n.c -lm
```